

```

1 public class Demo
2
3 {
4 public static void main(String[] args) {
5     byte b =127; //125;
6     int a = b;
7
8     system.out.println(a);
9 }
10 }
11
12
13
14     int a = 12;
15     byte k = (byte)a;
16
17     system.out.println(k);
18 }
19 }
20
21
22     int a = 257;
23     byte k = (byte) a;
24     float f = 5.6f;
25     int t =(int) f;
26     system.out.println(t);
27 }
28 }
29
30
31     byte = 10;
32     byte = 30;
33     int result = a*b;
34     system.out.println(result);
35 }
36 }
37
38 9. Arithmetic Operator
39
40
41 1. Assignment Operator
42 public class Arithmetic
43 {
44     public static void main(String[] args){
45         int num1 =7;
46         int num2 = 5;
47
48         int result = num1 + num2;
49         system.out.println(result);
50     }
51 }
52
53     int result = num1-num2;
54
55
56     int result = num1*num2;
57
58     int result = num1/num2;
59
60     int result = num1%num2
61
62
63     // int num1 = 7;
64     // num1 = num2+2;
65
66     // num1 +=2;
67     // num1 +=1;

```

```

68      // num1++; // post- Increment
69      ++num1;    // pre- Increment
70      // num1--; // Decrement
71      t result = num++; // fetch the value and then increment f
72      system.out.println(num1);
73  }
74  }
75
76
77
78  * Relational Operator
79
80  public class Relational
81  {
82      public static void main(String[] args){
83
84          int x = 6; // double x = 8.8;
85          int y = 5; // double y = 9.8;
86
87          // boolean result = x < y;
88
89          // boolean result = x !=y;
90          boolean result = x == y;
91
92
93          system.out.println(result);
94      }
95  }
96
97
98  *LOGICAL OPERATOR IN JAVA
99
100      combine two different logical operator in between
101
102      x < y  And/OR   a >b
103
104      T/F          T/F
105
106      &- And      |- OR          !- Not
107
108      T T - T      T T -T        T - F
109
110      T F - F      T F - T        F - T
111      F T - F      F T - T
112      F F - F      F F - F
113
114      &  |  !
115      && || !
116
117      r = x > y && a < b
118      T      T      T
119
120      x = x > y || a < b
121      T      T      T
122
123      r = x > y && a < b
124      T      F      T
125
126      example:-
127
128      public class Demo
129      {
130          public static void main (String [] args){
131
132              int x = 7;
133              int y = 5;
134              int a = 5;

```

```

135         int b = 9;
136
137         // boolean result = x > y ;
138         // boolean result = x > y && a > b;
139         // boolean result = x > y || a > b;
140         boolean result = x > y || a < b || a > 1;
141         boolean result = a > b;
142
143         System.out.println(result);
144     }
145 }
146
147
148 * IF Else in java
149
150 public class ifElse
151 {
152     public static void main(String[]args){
153
154         int x = 8;
155         int y = 7; // 17
156         int z = 5; // int z = 9;
157
158         if(x>y && x>z)
159             System.out.println(x);
160         else if(y>x && y>z)
161             System.out.println(y);
162         else
163             System.out.println(z);
164     }
165 }
166
167

```

14. Ternary Operator in java

```

168
169
170 public class ifElse
171 {
172     public static void main(String[]args){
173
174         int n = 4; // 5;
175         int result = 0;
176
177         if (n%2==0)
178             result = 10;
179         else
180             result = 20;
181
182         system.out.println(result);
183     }
184
185     }
186
187

```

2nd Method

```

188
189
190
191 public class ifElse
192 {
193     public static void main(String[]args){
194
195         int n = 4;
196         int result = 0;
197
198
199         // ?:
200
201         result = n%2==0 ? 10 : 20;

```

```
202         system.out.println(result);
203     }
204 }
205
206
```

207 15. Switch Statement in java

208
209 Switch Statement :- switch is multiple choice decision making selection statement, it used when we want to select only one case out of multiple cases.

```
210
211
212 public class SwitchStatement
213 {
214     public Static void main(String[]args){
215
216         int n = 1;
217
218         System.out.println("Monday");
219         System.out.println("Tuesday");
220         System.out.println("Wednesday");
221         System.out.println("Thursday");
222         System.out.println("Friday");
223         System.out.println("Saturday");
224         System.out.println("Sunday");
225
226     }
227 }
228
229 // public class SwitchStatement
230 {
231     public Static void main(String[]args){
232
233         int n = 2; // int n = 8;
234
235         switch (n){
236
237         case 1:
238             System.out.println("Monday");
239             break;
240         case 2:
241             System.out.println("Tuesday");
242             break;
243         case 3:
244             System.out.println("Wednesday");
245             break;
246         case 4:
247             System.out.println("Thursday");
248             break;
249         case 5:
250             System.out.println("Friday");
251             break;
252         case 6:
253             System.out.println("Saturday");
254             break;
255         case 7:
256             System.out.println("Sunday");
257             break;
258         default:
259             System.out.println("Enter a default number");
260
261     }
262 }
263
264
```

265 #. What's new in java switch Statement and Expression

```
266
267 public class switch
```

```

268     {
269
270     public static void main(String []args){
271
272         string day = "Monday";
273
274         switch(day)
275         {
276             case "Satuday", "Sunday":
277                 System.out.println("6am");
278                 break;
279             case "Monday":
280                 System.out.println("8am");
281
282             default:
283                 System.out.println("7am");
284             }
285         }
286
287

```

16. Need For Loop in java

For Loop:- for loop is the most commonly used loop. it is used when we want to perform initialization, condition and incr|decr operation in single line

```

291
292
293     package Loop;
294
295     public class Loop {
296     public static void main(String[]args) {
297         System.out.println("hi");
298         System.out.println("hi");
299         System.out.println("hi");
300         System.out.println("hi");
301     }
302
303 }
304
305 package Loop;
306
307 public class Loop {
308     public static void main(String[]args) {
309
310         //repeat this statement 4time
311         // loop - while loop,do while loop,For loop
312
313         // 100 - condition
314         System.out.println("hi");
315     }
316
317 }
318

```

17. While Loop in java

While Loop:-while loop is a pre-test loop,ti is used when we dont know the no. of iterations in advance.

It is also known as entry control loop.

```

322
323
324     package WhileLoop;
325
326     public class WhileLoop {
327     public static void main(String[]args) {
328
329         int i = 1;
330
331         while(i<=6)
332         {

```

```

333         System.out.println("hi"+ i);
334
335         i++;
336     }
337 }
338
339 }
340
341 ## package WhileLoop;
342
343 public class WhileLoop {
344     public static void main(String[]args) {
345
346         int i = 1;
347
348         while(i<= 6)
349         {
350             System.out.println("hi"+ i);
351
352             i++;
353         }
354         System.out.println("Bye" + i);
355     }
356 }
357
358
359 ## package WhileLoop;
360
361 public class WhileLoop {
362     public static void main(String[]args) {
363
364         int i = 1;
365
366         while(i<=4)
367         {
368             System.out.println("hi"+ i);
369
370             int j = 1;
371
372             while(j<=3) {
373                 System.out.println("Hello" +j);
374                 j++;
375             }
376
377             i++;
378         }
379         System.out.println("Bye" + i);
380     }
381 }
382
383 }
384
385
386

```

18. Do-While Loop in java

Do-While Loop:-is a post-test loop,it is used when we want to execute loop body atleast once even condition is false.

It is also known as exit control loop.

```

390 package DoWhile;
391
392 public class DoWhile {
393     public static void main(String[]args) {
394
395
396         int i = 1;
397
398

```

```

399         while(i<=4)
400         {
401
402             System.out.println("Hello" +i);
403
404             i++;
405         }
406     }
407
408 }
409
410
411 package DoWhile;
412
413 public class DoWhile {
414     public static void main(String[]args) {
415
416
417         int i = 1;
418
419         do
420         {
421
422             System.out.println("Hello" +i);
423
424             i++;
425         }while(i<=4);
426     }
427 }
428
429
430 19.For-Loop
431
432 package ForLoop;
433
434 public class ForLoop {
435     public static void main(String[]args) {
436
437
438
439         for(int i=1; i<=4;i++) // increment
440         {
441             System.out.println("hi" +i);
442         }
443     }
444 }
445
446 }
447
448 ## package Array;
449
450 public class Array {
451
452     public static void main(String[]args)
453     {
454         int i, j;
455         for(i=1;i<=5; i++)
456         {
457             for(j=1;j<=i;j++)
458             {
459                 System.out.print(" * ");
460             }
461             System.out.print("\n");
462         }
463     }
464 }
465

```

```
466
467 package ForLoop;
468
469 public class ForLoop {
470     public static void main(String[]args) {
471
472         //int i = 1;
473
474         for(int i=4; i>=1; i--) //decrement
475         {
476             System.out.println("hi" +i);
477         }
478     }
479 }
480
481 }
482
483 ## package ForLoop;
484
485 public class ForLoop {
486     public static void main(String[]args) {
487
488         //int i = 1;
489
490         for(int i=0; i<=3; i++)
491         {
492             System.out.println("hi" +i);
493         }
494     }
495 }
496
497 }
498
499 ## less than
500 package ForLoop;
501
502 public class ForLoop {
503     public static void main(String[]args) {
504
505         //int i = 1;
506
507         for(int i=0; i<4; i++)
508         {
509             System.out.println("hi" +i);
510         }
511     }
512 }
513
514 }
515
516 ## package ForLoop;
517
518 public class ForLoop {
519     public static void main(String[]args) {
520
521         //int i = 1;
522
523         for(int i=1; i<=7; i++)
524         {
525             System.out.println("Day" +i);
526         }
527     }
528 }
529
530 }
531
532
```



```

533  ## package ForLoop;
534
535  public class ForLoop {
536      public static void main(String[]args) {
537
538          //int i = 1;
539
540          for(int i=1; i<=5; i++)
541          {
542              System.out.println("Day" +i);
543
544              for(int j=1; j<=9; j++)
545              {
546
547
548
549                  System.out.println("    9");
550              }
551          }
552
553      }
554
555  }
556
557
558  ## package ForLoop;
559
560  public class ForLoop {
561      public static void main(String[]args) {
562
563          //int i = 1;
564
565          for(int i=1; i<=5; i++)
566          {
567              System.out.println("Day" +i);
568
569              for(int j=1; j<=9; j++)
570              {
571
572
573
574                  System.out.println("    " +j);
575              }
576          }
577
578      }
579
580  }
581
582  ## package ForLoop;
583
584  public class ForLoop {
585      public static void main(String[]args) {
586
587          //int i = 1;
588
589          for(int i=1; i<=5; i++)
590          {
591              System.out.println("Day" +i);
592
593              for(int j=1; j<=9; j++)
594              {
595
596
597
598                  System.out.println("    " +(j+8));
599              }

```

```

600         }
601     }
602 }
603
604 }
605
606 ## package ForLoop;
607
608 public class ForLoop {
609     public static void main(String[]args) {
610
611         //int i = 1;
612
613         for(int i=1; i<=5; i++)
614         {
615             System.out.println("Day" +i);
616
617             for(int j=1; j<=9; j++)
618             {
619
620
621
622                 System.out.println(" " +(j+8) + " - " + (j+9));
623             }
624         }
625
626     }
627 }
628 }
629
630
631 ## package ForLoop;
632
633 public class ForLoop {
634     public static void main(String[]args) {
635
636         int i = 1;
637
638         for(; i<=5;)
639         {
640             System.out.println("Day" +i);
641
642             // for(int j=1; j<=9; j++)
643             // {
644
645
646
647             // System.out.println(" " +(j+8) + " - " + (j+9));
648             // }
649             i++;
650         }
651
652     }
653 }
654 }
655
656

```

21.Class And Object Theory in java

Class:- class is a collection of objects and it doesnt take any space on memory,class is also called as blueprint/ logical entity.

Object:-object is an instace of class that execute the class. onec the object is created, it takes up space like other variable in momory.

syntax:- class-name obj-name = new class-name

```

665 // Object Oriented Programing
666
667 // Object- Properties and Behaviors
668
669 first create:----
670 // class
671 give the design in class file you have a class files which a buluprint
672
673
674
675 22.Class and Object Practical in java
676
677 package Object;
678
679 public class Object {
680
681     public static void main(String[]args)
682     {
683
684         int num1 = 4;
685         int num2 = 5;
686
687         int result = num1 + num2;
688
689         System.out.println(result);
690
691     }
692
693 }
694
695 ## package Object;
696
697 public class Eample {
698
699     public static void main(String[]args) {
700         System.out.println("Hello class");
701     }
702
703
704     public static void get(String[]args) {
705         System.out.println("Hello class");
706     }
707
708 }
709
710 ## package Object;
711
712 public class Eample {
713
714
715
716     public static void main(String[]args) {
717         System.out.println("Hello Mantu");
718     }
719
720
721 }
722
723
724 class User{
725     public int data =10;
726     public String name ="Mantu";
727
728     public static void get(String[]args) {
729         System.out.println("Hello class");
730     }
731

```

```

732     }
733
734
735
736     ## package Object;
737
738     public class Eample {
739
740
741
742         public static void main(String[]args) {
743             User U1 = new User();
744             System.out.println("Hello ");
745
746         }
747
748     }
749
750
751     class User{
752         public int data =10;
753         public String name ="Mantu";
754
755         public static void get(String[]args) {
756             System.out.println("Hello class");
757         }
758     }
759
760     ## package Object;
761
762
763
764     class demo{
765         int a ;
766
767         public int add()
768         {
769             System.out.println("in add");
770             return 0;
771         }
772     }
773
774
775
776
777     public class Eample
778     {
779         public static void main(String[]args) {
780
781             int num1=4;
782             int num2=5;
783             int result = num1 + num2;
784             System.out.println(result);
785
786         }
787
788
789
790
791
792     }
793
794
795
796     23.JDK JRE JVM in java
797
798

```

```
799
800
801 24.Method in java
802
803
804 package Calculator;
805
806
807 class Computer
808 {
809
810     public void playMusic()
811     {
812         System.out.println("Music playing...");
813     }
814
815     public String getMeAPen(int cost)
816     {
817         return "Pen";
818     }
819 }
820
821 public class Calculator {
822
823     public static void main(String[]args) {
824
825         Computer obj = new Computer();
826         obj.playMusic();
827     }
828 }
829
830
831 ## package Calculator;
832
833
834 class Computer
835 {
836
837     public void playMusic()
838     {
839         System.out.println("Music playing...");
840     }
841
842     public String getMeAPen(int cost)
843     {
844         return "Pen";
845     }
846 }
847
848 public class Calculator {
849
850     public static void main(String[]args) {
851
852         Computer obj = new Computer();
853         obj.playMusic();
854
855         String str =obj.getMeAPen( 10);
856         System.out.println(str);
857     }
858 }
859
860
861 ## package Calculator;
862
863
864 class Computer
865 {
```

```

866
867     public void playMusic()
868     {
869         System.out.println("Music playing...");
870     }
871
872     public String getMeAPen(int cost)
873     {
874         if(cost >= 10)
875             return "Pen";
876         else
877             return "Nothing";
878     }
879 }
880
881 public class Calculator {
882
883     public static void main(String[]args) {
884
885         Computer obj = new Computer();
886         obj.playMusic();
887
888         String str =obj.getMeAPen( 2);
889         System.out.println(str);
890     }
891 }
892
893
894 ## package Calculator;
895
896
897 class Computer
898 {
899
900     public void playMusic()
901     {
902         System.out.println("Music playing...");
903     }
904
905     public String getMeAPen(int cost)
906     {
907         if(cost >= 10)
908             return "Pen";
909
910         return "Nothing";
911     }
912 }
913
914 public class Calculator {
915
916     public static void main(String[]args) {
917
918         Computer obj = new Computer();
919         obj.playMusic();
920
921         String str =obj.getMeAPen( 12);
922         System.out.println(str);
923     }
924 }

```

25. Method Overloading in java

```

package Calculator;

```

```

933 class School
934 {
935     public int add(int n1, int n2)
936     {
937
938         return n1 + n2;
939     }
940
941 }
942
943
944
945 public class Calculator {
946
947     public static void main(String[]args) {
948         School obj = new School();
949         int r1 = obj.add ( 3, 4);
950         System.out.println(r1);
951     }
952
953 }
954
955
956 ## Three nubmer add
957
958 package Calculator;
959
960
961 class School
962 {
963     public int add(int n1, int n2 ,int n3)
964     {
965
966         return n1 +    n2 + n3 ;
967     }
968
969 }
970
971
972
973 public class Calculator {
974
975     public static void main(String[]args) {
976         School obj = new School();
977         int r1 = obj.add ( 3, 4 , 9);
978         System.out.println(r1);
979     }
980
981 }
982
983 ## package Calculator;
984
985
986 class School
987 {
988     public int add(int n1, int n2 ,int n3)
989     {
990
991         return n1 +    n2 + n3 ;
992     }
993
994     public int  add(int n1, int n2)
995     {
996         return n1+ n2;
997     }
998     public double  add(double n1, int n2)
999     {

```

```
1000         return n1 + n2;
1001     }
1002 }
1003
1004
1005
1006 public class Calculator {
1007
1008     public static void main(String[]args) {
1009         School obj = new School();
1010         int r1 = obj.add ( 3, 4 );
1011         System.out.println(r1);
1012     }
1013
1014 }
```

1016
1017 26. Stack And Heap in java?
1018

```
1019 package Calculator;
1020
1021 class School
1022 {
1023     int num;
1024     public int add(int n1, int n2 )
1025     {
1026
1027         return n1 +    n2    ;
1028     }
1029
1030 }
1031
1032 public class Calculator {
1033
1034     public static void main(String[]args) {
1035         int data = 10;
1036
1037         School obj = new School();
1038         int r1 = obj.add ( 3, 4 );
1039         System.out.println(r1);
1040     }
1041
1042 }
1043
1044 ## package Calculator;
1045
1046
1047 class School
1048 {
1049     int num = 5;
1050     public int add(int n1, int n2 )
1051     {
1052         System.out.println(num);
1053         return n1 +    n2    ;
1054     }
1055
1056 }
1057 public class Calculator {
1058
1059     public static void main(String[]args) {
1060         int data = 10;
1061
1062         School obj = new School();
1063         int r1 = obj.add ( 3, 4 );
1064         System.out.println(r1);
1065     }
1066 }
```



```

1067     }
1068
1069     ## package Calculator;
1070
1071
1072     class School
1073     {
1074         int num=5;
1075         public int add(int n1, int n2 )
1076         {
1077             // System.out.println(num);
1078             return n1 + n2 ;
1079         }
1080
1081     }
1082
1083     public class Calculator {
1084
1085         public static void main(String[]args) {
1086             int data = 10;
1087
1088             School obj = new School();
1089             School obj1 = new School();
1090             int r1 = obj.add ( 3, 4 );
1091             // System.out.println(r1);
1092             System.out.println(obj.num);
1093             System.out.println(obj1.num);
1094         }
1095     }
1096
1097
1098
1099     ## package Calculator;
1100
1101
1102     class School
1103     {
1104         int num=5;
1105         public int add(int n1, int n2 )
1106         {
1107             // System.out.println(num);
1108             return n1 + n2 ;
1109         }
1110
1111     }
1112
1113
1114
1115     public class Calculator {
1116
1117         public static void main(String[]args) {
1118             int data = 10;
1119
1120             School obj = new School();
1121             School obj1 = new School();
1122             int r1 = obj.add ( 3, 4 );
1123             // System.out.println(r1);
1124
1125             obj.num = 8;
1126
1127             System.out.println(obj.num);
1128             System.out.println(obj1.num);
1129         }
1130     }
1131
1132
1133     27. Need of an Array in java

```

```

1134
1135 Array:- in a variable can store an value type:-
1136 Integer,Falot,Double,String,Boolean
1137
1138 example:- int i = 5;
1139           int j = 6;
1140           int k = 7;
1141
1142           int num [] ={5,6,7};
1143
1144           int num1[] = new int[4];
1145
1146           Index number
1147           [3][7][2][4]
1148           0   1   2   3
1149
1150
1151           ## package Array;
1152
1153 public class Array {
1154
1155     public static void main(String[]args)
1156     {
1157         int nums[] = {3,7,4};
1158         System.out.println(nums[0]);
1159     }
1160 }
1161
1162
1163 ## package Array;
1164
1165 public class Array {
1166
1167     public static void main(String[]args)
1168     {
1169         int nums[] = {3,7,4, 9};
1170         System.out.println(nums[1]);
1171     }
1172 }
1173
1174
1175 ## package Array;
1176
1177 public class Array {
1178
1179     public static void main(String[]args)
1180     {
1181         int nums[] = {3,7,4, 9};
1182         nums[1]=6;
1183
1184
1185         System.out.println(nums[1]);
1186     }
1187 }
1188
1189
1190
1191 ## package Array;
1192
1193 public class Array {
1194
1195     public static void main(String[]args)
1196     {
1197         int nums[] = new int[4];
1198         //  nums[1]=6;
1199
1200

```

```

1201
1202         System.out.println(nums[1]);
1203     }
1204
1205 }
1206 // index no:-0
1207
1208 ## package Array;
1209
1210 public class Array {
1211
1212     public static void main(String[]args)
1213     {
1214         int nums[] = new int[4];
1215         nums[0]=4;
1216         nums[1]=8;
1217         nums[2]=3;
1218         nums[3]=9;
1219
1220         System.out.println(nums[0]);
1221         System.out.println(nums[1]);
1222         System.out.println(nums[2]);
1223         System.out.println(nums[3]);
1224     }
1225
1226 }
1227
1228 ## package Array;
1229
1230 public class Array {
1231
1232     public static void main(String[]args)
1233     {
1234         int nums[] = new int[4];
1235         nums[0]=4;
1236         nums[1]=8;
1237         nums[2]=3;
1238         nums[3]=9;
1239
1240         for(int i =0; i<=3; i++)
1241         {
1242             System.out.println(nums[i]);
1243         }
1244     }
1245 }
1246
1247
1248
1249 29.Multi Dimensional Array in java
1250
1251 package Array;
1252
1253 public class Array {
1254
1255     public static void main(String[]args)
1256     {
1257         int nums[][] = new int[3][4];
1258
1259         for(int i=0;i<3;i++)
1260         {
1261
1262             for(int j=0;j<4;j++)
1263             {
1264                 System.out.print(nums[i][j] + " ");
1265             }
1266             System.out.println();
1267         }

```

```
1268
1269
1270     }
1271 }
1272
1273 ## package Array;
1274
1275 public class Array {
1276
1277     public static void main(String[]args)
1278     {
1279         int nums[][] = new int[3][4];
1280
1281
1282         for(int i=0;i<3;i++)
1283         {
1284
1285             for(int j=0;j<4;j++)
1286             {
1287                 nums[i][j] = (int) (Math.random() *100);
1288                 System.out.println(nums[i][j]);
1289             }
1290
1291         }
1292
1293
1294
1295
1296         for(int i=0;i<3;i++)
1297         {
1298
1299             for(int j=0;j<4;j++)
1300             {
1301                 System.out.print(nums[i][j] + " ");
1302             }
1303             System.out.println();
1304         }
1305
1306
1307     }
1308 }
1309
1310 ## package Array;
1311
1312 public class Array {
1313
1314     public static void main(String[]args)
1315     {
1316         int nums[][] = new int[3][4];
1317
1318
1319         for(int i=0;i<3;i++)
1320         {
1321
1322             for(int j=0;j<4;j++)
1323             {
1324                 nums[i][j] = (int) (Math.random() *100);
1325
1326             }
1327
1328         }
1329
1330
1331
1332
1333         for(int i=0;i<3;i++)
1334         {
```

```
1335
1336         for(int j=0;j<4;j++)
1337         {
1338             System.out.print(nums[i][j] + " ");
1339         }
1340         System.out.println();
1341     }
1342
1343
1344     }
1345 }
1346
1347 ## package Array;
1348
1349 public class Array {
1350
1351     public static void main(String[]args)
1352     {
1353         int nums[][] = new int[3][4];
1354
1355
1356         for(int i=0;i<3;i++)
1357         {
1358
1359             for(int j=0;j<4;j++)
1360             {
1361                 nums[i][j] = (int)(Math.random() *10);
1362             }
1363         }
1364
1365
1366
1367
1368
1369
1370         for(int i=0;i<3;i++)
1371         {
1372
1373             for(int j=0;j<4;j++)
1374             {
1375                 System.out.print(nums[i][j] + " ");
1376             }
1377             System.out.println();
1378         }
1379
1380     }
1381 }
1382
1383
1384
1385 ## package Array;
1386
1387 public class Array {
1388
1389     public static void main(String[]args)
1390     {
1391         int nums[][] = new int[3][4];
1392
1393
1394         for(int i=0;i<3;i++)
1395         {
1396
1397             for(int j=0;j<4;j++)
1398             {
1399                 nums[i][j] = (int)(Math.random() *10);
1400             }
1401         }
```

```

1402
1403     }
1404
1405
1406
1407
1408     for(int i=0;i<3;i++)
1409     {
1410
1411         for(int j=0;j<4;j++)
1412         {
1413             System.out.print(nums[i][j] + " ");
1414         }
1415         System.out.println();
1416     }
1417
1418     for(int n[] : nums)
1419     {
1420         for(int m : n)
1421         {
1422             System.out.println(m+ "");
1423         }
1424         System.out.println();
1425     }
1426
1427
1428     }
1429 }
1430
1431 ## package Array;
1432
1433 public class Array {
1434
1435     public static void main(String[]args)
1436     {
1437         int nums[][] = new int[3][4]; // jagged Array
1438
1439         nums[0]= new int[3];
1440         nums[1]= new int[4];
1441         nums[2]= new int[2];
1442
1443         for(int i=0;i<nums.length;i++)
1444         {
1445
1446             for(int j=0;j<nums[i].length;j++)
1447             {
1448                 nums[i][j] = (int) (Math.random() *10);
1449
1450             }
1451
1452         }
1453
1454
1455
1456         for(int n[]: nums)
1457         {
1458             for(int m: n)
1459             {
1460                 System.out.print(m + " ");
1461             }
1462             System.out.println();
1463
1464         }
1465     }
1466 }

```

```
1469 package Array;
1470
1471 public class Array
1472 {
1473
1474     public static void main(String[]args)
1475     {
1476
1477         int nums[] = new int[4];
1478         nums[0] = 4;
1479         nums[1] = 8;
1480         nums[2] = 3;
1481         nums[3] = 9;
1482
1483         for(int i=0;i<4;i++)
1484         {
1485             System.out.println(nums[i]);
1486         }
1487     }
1488 }
1489
1490
```

32.Array of Object in java

```
1491
1492
1493 package Array;
1494
1495 public class Array
1496 {
1497
1498     public static void main(String[]args)
1499     {
1500
1501         int nums[] = new int[6];
1502         nums[0] = 4;
1503         nums[1] = 8;
1504         nums[2] = 3;
1505         nums[3] = 9;
1506
1507         for(int i=0;i<6;i++)
1508         {
1509             System.out.println(nums[i]);
1510         }
1511     }
1512 }
1513
1514 ## package Array;
1515
1516 public class Array
1517 {
1518
1519     public static void main(String[]args)
1520     {
1521
1522         int nums[] = new int[6];
1523         nums[0] = 4;
1524         nums[1] = 8;
1525         nums[2] = 3;
1526         nums[3] = 9;
1527
1528         for(int i=0;i<nums.length;i++)
1529         {
1530             System.out.println(nums[i]);
1531         }
1532     }
1533 }
1534
1535 ## package Array;
```

```

1536
1537 class Students
1538 {
1539     int rollno;
1540     String name;
1541     int marks;
1542 }
1543
1544 public class Array
1545 {
1546
1547     public static void main(String[]args)
1548     {
1549         Students s1 = new Students();
1550         s1.rollno = 1;
1551         s1.name = "Mantu";
1552         s1.marks = 96;
1553
1554         Students s2 = new Students();
1555         s2.rollno = 2;
1556         s2.name = "Karan";
1557         s2.marks = 94;
1558
1559         Students s3 = new Students();
1560         s3.rollno = 3;
1561         s3.name = "Aman";
1562         s3.marks = 88;
1563
1564         System.out.println(s2.name + " : " + s2.marks);
1565
1566         Students students[] = new Students[3];
1567         students[0] = s1;
1568         students[1] = s2;
1569         students[2] = s3;
1570
1571     }
1572 }
1573
1574
1575

```

1576 33.Enhanced for Loop in java

```

1577
1578 package Array;
1579
1580
1581 public class Array
1582 {
1583
1584     public static void main(String[]args)
1585     {
1586         int nums[] = new int [4];
1587
1588         nums[0] = 4;
1589         nums[1] = 8;
1590         nums[2] = 3;
1591         nums[3] = 9;
1592
1593         for(int n : nums)
1594         {
1595             System.out.println(n);
1596         }
1597     }
1598 }
1599

```

1600 34.What is String in java

1602


```
1603 package Array;
1604
1605
1606 public class Array
1607 {
1608
1609     public static void main(String[]args)
1610     {
1611         String name = new String( "Mantu");
1612
1613         System.out.println(name);
1614     }
1615 }
1616
1617 ## package Array;
1618
1619
1620 public class Array
1621 {
1622
1623     public static void main(String[]args)
1624     {
1625         String name = new String( "mantu");
1626
1627         System.out.println(name);
1628         System.out.println(name.hashCode());
1629     }
1630 }
1631
1632 ## package Array;
1633
1634
1635 public class Array
1636 {
1637
1638     public static void main(String[]args)
1639     {
1640         String name = new String( "mantu");
1641
1642         System.out.println("hello" + name );
1643     }
1644 }
1645
1646
1647 ## package Array;
1648
1649
1650 public class Array
1651 {
1652
1653     public static void main(String[]args)
1654     {
1655         String name = new String( "mantu");
1656
1657         System.out.println("hello" + name );
1658         System.out.println(name.charAt(1));
1659     }
1660 }
1661
1662 ## package Array;
1663
1664
1665 public class Array
1666 {
1667
1668     public static void main(String[]args)
1669     {
```

```

1670         String name = new String( "mantu");
1671
1672         System.out.println("hello" + name );
1673         System.out.println(name.concat("kumar"));
1674     }
1675 }
1676
1677 ## package Array;
1678
1679
1680 public class Array
1681 {
1682
1683     public static void main(String[]args)
1684     {
1685         String name = "mantu";
1686
1687         System.out.println("hello" + name );
1688
1689     }
1690 }
1691
1692 35.Mutable vs immutable String in java
1693     package Array;
1694
1695
1696 public class Array
1697 {
1698
1699     public static void main(String[]args)
1700     {
1701         String name = " mantu";
1702         name = name+ " kumar ";
1703         System.out.println(" hello" + name );
1704
1705     }
1706 }
1707
1708
1709 36.StringBuffer and StringBuilder in java
1710
1711 . An objects of string class are immutable, objects of
1712 stringBuffer and stringBuilder class are mutable.
1713
1714
1715
1716     package Array;
1717
1718
1719
1720 public class Array
1721 {
1722
1723     public static void main(String[]args)
1724     {
1725         StringBuffer sb = new StringBuffer("Mantu");
1726         System.out.println(sb.capacity());
1727     }
1728 }
1729
1730
1731 ## package Array;
1732
1733
1734 public class Array
1735 {
1736

```

```
1737     public static void main(String[]args)
1738     {
1739         StringBuffer sb = new StringBuffer("Mantu");
1740         System.out.println(sb.length());
1741     }
1742 }
1743
1744
1745 ## package Array;
1746
1747 public class Array
1748 {
1749
1750     public static void main(String[]args)
1751     {
1752         StringBuffer sb = new StringBuffer("Mantu");
1753         sb.append("kumar");
1754
1755         System.out.println(sb);
1756     }
1757 }
1758
1759 ## package Array;
1760
1761 public class Array
1762 {
1763     public static void main(String[]args)
1764     {
1765         StringBuffer sb = new StringBuffer("Mantu");
1766         sb.append("kumar");
1767
1768         sb.deleteCharAt(2); // deleteCharAt
1769
1770         System.out.println(sb);
1771     }
1772 }
1773
1774 ## package Array;
1775
1776 public class Array
1777 {
1778     public static void main(String[]args)
1779     {
1780         StringBuffer sb = new StringBuffer("Mantu");
1781         sb.append("kumar");
1782
1783         sb.insert(0, "java "); // insert
1784
1785         System.out.println(sb);
1786     }
1787 }
1788
1789 ## package Array;
1790
1791 public class Array
1792 {
```

```

1804
1805     public static void main(String[]args)
1806     {
1807         StringBuffer sb = new StringBuffer("Mantu");
1808         sb.append("kumar");
1809
1810         sb.insert(6, "java ");
1811         sb.setLength(30);
1812
1813
1814         System.out.println(sb);
1815
1816     }
1817 }
1818
1819
1820     37. String Variable in java
1821
1822 package Mantu;
1823
1824 class Mobile
1825 {
1826     String brand;
1827     int price;
1828     String name;
1829
1830     public void show()
1831     {
1832         System.out.println(brand +": "+ price +": "+ name);
1833     }
1834 }
1835
1836 public class Mantu {
1837     public static void main(String[]args)
1838     {
1839         Mobile obj1 = new Mobile();
1840         obj1.brand = "Apple";
1841         obj1.price = 1500;
1842         obj1.name = "Smart Phone";
1843
1844         Mobile obj2 = new Mobile();
1845         obj2.brand = "Nokia";
1846         obj2.price = 1200;
1847         obj2.name = "Smart Phone";
1848
1849         obj1.show();
1850         obj2.show();
1851     }
1852 }
1853
1854
1855 ## CHANGE THE NAME:-
1856 package Mantu;
1857
1858 class Mobile
1859 {
1860     String brand;
1861     int price;
1862     static String name;
1863
1864     public void show()
1865     {
1866         System.out.println(brand +": "+ price +": "+ name);
1867     }
1868 }
1869
1870

```

```

1871
1872     Mobile obj1 = new Mobile();
1873     obj1.brand = "Apple";
1874     obj1.price = 1500;
1875     obj1.name = "Smart Phone";
1876
1877     Mobile obj2 = new Mobile();
1878     obj2.brand = "Nokia";
1879     obj2.price = 1200;
1880     obj2.name = "Smart Phone";
1881
1882     obj1.name = "Phone";
1883
1884     obj1.show();
1885     obj2.show();
1886 }
1887
1888
1889 public class Mantu {
1890 public static void main(String[]args)
1891 {
1892     38.Static Method in java
1893     package Mantu;
1894
1895     class Mobile
1896     {
1897         String brand;
1898         int price;
1899         static String name;
1900
1901         public void show()
1902         {
1903             System.out.println(brand +": "+ price +": "+ name);
1904         }
1905
1906
1907     public static void show1(Mobile obj)
1908     {
1909         System.out.println(obj.brand + " : " + obj.price + " : " + name);
1910     }
1911 }
1912
1913
1914 public class Mantu {
1915 public static void main(String[]args)
1916 {
1917     Mobile obj1 = new Mobile();
1918     obj1.brand = "Apple";
1919     obj1.price = 1500;
1920     Mobile.name = "Smart Phone";
1921
1922     Mobile obj2 = new Mobile();
1923     obj2.brand = "Nokia";
1924     obj2.price = 1200;
1925     Mobile.name = "Smart Phone";
1926
1927     Mobile.name = "Phone";
1928
1929     obj1.show();
1930     obj2.show();
1931
1932     Mobile.show1(obj1);
1933 }
1934
1935
1936 39.Static Block in java
1937 package Mantu;

```

```

1938
1939 class Mobile
1940 {
1941     String brand;
1942     int price;
1943     static String name; // initialization
1944
1945
1946     public Mobile()
1947     {
1948         brand = "";
1949         price = 2000;
1950         System.out.println("in constructor"); // constructor
1951     }
1952     static
1953     {
1954         name = "Phone";
1955         System.out.println("in static block");
1956     }
1957
1958     public void show()
1959     {
1960         System.out.println(brand + ": " + price + ": " + name);
1961     }
1962
1963 }
1964
1965 public class Mantu {
1966     public static void main(String[] args)
1967     {
1968         Mobile obj1 = new Mobile();
1969         obj1.brand = "Apple";
1970         obj1.price = 1500;
1971         Mobile.name = "Smart Phone";
1972
1973         Mobile obj2 = new Mobile();
1974         obj2.brand = "Nokia";
1975         obj2.price = 1200;
1976         Mobile.name = "Smart Phone";
1977
1978         Mobile.name = "Phone";
1979
1980
1981
1982     }
1983 }
1984
1985
1986 40.Encapsulation in java
1987
1988 package Mantu;
1989
1990 class Human
1991 {
1992     int age;
1993     String name;
1994 }
1995 public class Mantu {
1996
1997     public static void main(String[] args)
1998     {
1999         Human obj = new Human();
2000         obj.age = 15;
2001         obj.name = "Mantu";
2002
2003         System.out.println(obj.name);
2004     }

```

```
2005
2006 }
2007
2008
2009 ## package Mantu;
2010
2011 class Human
2012 {
2013     private int age = 11;
2014     private String name = "Mantu";
2015
2016     public int getAge()
2017     {
2018         return age;
2019     }
2020
2021     public String getName()
2022     {
2023         return name;
2024     }
2025 }
2026 public class Mantu {
2027
2028     public static void main(String[]args)
2029     {
2030         Human obj = new Human();
2031         //obj.age = 15;
2032         // obj.name = "Mantu";
2033
2034         System.out.println(obj.getName() + " : " + obj.getAge());
2035     }
2036
2037 }
2038
2039 ## package Mantu;
2040
2041 class Human
2042 {
2043     private int age;
2044     private String name;
2045
2046     public int getAge()
2047     {
2048         return age;
2049     }
2050
2051     public String getName()
2052     {
2053         return name;
2054     }
2055 }
2056 public class Mantu {
2057
2058     public static void main(String[]args)
2059     {
2060         Human obj = new Human();
2061         //obj.age = 15;
2062         // obj.name = "Mantu";
2063
2064         System.out.println(obj.getName() + " : " + obj.getAge());
2065     }
2066
2067 }
2068
2069 ## package Mantu;
2070
2071 class Human
```

```

2072 {
2073     private int age;
2074     private String name;
2075
2076     public int getAge()
2077     {
2078         return age;
2079     }
2080     public void setAge(int a)
2081     {
2082         age = a;
2083     }
2084     public String getName()
2085     {
2086         return name;
2087     }
2088     public void setName(String n )
2089     {
2090         name = n;
2091     }
2092 }
2093 public class Mantu {
2094
2095     public static void main(String[]args)
2096     {
2097         Human obj = new Human();
2098         obj.setAge(30);
2099         obj.setName("Mantu");
2100
2101         System.out.println(obj.getName() + " : " + obj.getAge());
2102     }
2103
2104 }

```

2106 41. Getters and Setters in java

2109 42.This Keyword in java

```

2110 package Mantu;
2111
2112 class Human
2113 {
2114     private int age;
2115     private String name;
2116
2117     public int getAge()
2118     {
2119         return age;
2120     }
2121     public void setAge(int a)
2122     {
2123         age = a;
2124     }
2125     public String getName()
2126     {
2127         return name;
2128     }
2129     public void setName(String n )
2130     {
2131         name = n;
2132     }
2133 }
2134 public class Mantu {
2135
2136     public static void main(String[]args)
2137     {
2138         Human obj = new Human();

```



```

2139         obj.setAge(30);
2140         obj.setName("Mantu");
2141
2142         System.out.println(obj.getName() + " : " + obj.getAge());
2143     }
2144
2145 }
2146
2147 ## package Mantu;
2148
2149 class Human
2150 {
2151     private int age;
2152     private String name;
2153
2154     public int getAge()
2155     {
2156         return age;
2157     }
2158     public void setAge(int age)
2159     {
2160         Human obj1 = new Human();
2161         obj1.age = age;
2162     }
2163     public String getName()
2164     {
2165         return name;
2166     }
2167     public void setName(String n)
2168     {
2169         name = n;
2170     }
2171 }
2172 public class Mantu {
2173
2174     public static void main(String[] args)
2175     {
2176         Human obj = new Human();
2177         obj.setAge(30);
2178         obj.setName("Mantu");
2179
2180         System.out.println(obj.getName() + " : " + obj.getAge());
2181     }
2182
2183 }
2184
2185 ## package Mantu;
2186
2187 class Human
2188 {
2189     private int age;
2190     private String name;
2191
2192     public int getAge()
2193     {
2194         return age;
2195     }
2196     public void setAge(int age, Human obj)
2197     {
2198         Human obj1 = obj;
2199         obj1.age = age;
2200     }
2201     public String getName()
2202     {
2203         return name;
2204     }
2205     public void setName(String n)

```

```
2206     {
2207         name = n;
2208     }
2209 }
2210 public class Mantu {
2211
2212     public static void main(String[]args)
2213     {
2214         Human obj = new Human();
2215         obj.setAge(30,obj);
2216         obj.setName("Mantu");
2217
2218         System.out.println(obj.getName() + " : " + obj.getAge());
2219     }
2220
2221 }
2222
2223
2224 43.Constructor in java
2225
2226 package Mantu;
2227
2228 class Human
2229 {
2230     private int age;
2231     private String name;
2232
2233     // no -argument constructor
2234     public Human()
2235     {
2236         age = 12;
2237         name = "Aman";
2238     }
2239
2240
2241
2242     public int getAge()
2243     {
2244         return age;
2245     }
2246     public void setAge(int age)
2247     {
2248
2249         this.age = age;
2250     }
2251     public String getName()
2252     {
2253         return name;
2254     }
2255     public void setName(String name)
2256     {
2257         this.name = name;
2258     }
2259 }
2260 public class Mantu {
2261
2262     public static void main(String[]args)
2263     {
2264         // calling no -argument constructor
2265         Human obj = new Human();
2266         Human obj1 = new Human();
2267         System.out.println(obj.getName() + " : " + obj.getAge());
2268
2269
2270         //obj.setAge(30);
2271         //obj.setName("Mantu");
2272
```

```

2273 // System.out.println(obj.getName() + " : " + obj.getAge());
2274 }
2275
2276 }
2277
2278
2279 44. Default vs Parameterized Constructor in java
2280
2281 package Mantu;
2282
2283 class Human
2284 {
2285     private int age;
2286     private String name;
2287
2288     // default constructor
2289     public Human()
2290     {
2291         age = 12;
2292         name = "Aman";
2293     }
2294     // parameterized constructor
2295     public Human(int a, String n)
2296     {
2297         age =a;
2298         name = n;
2299     }
2300
2301     public int getAge()
2302     {
2303         return age;
2304     }
2305     public void setAge(int age)
2306     {
2307
2308         this.age = age;
2309     }
2310     public String getName()
2311     {
2312         return name;
2313     }
2314     public void setName(String name)
2315     {
2316         this.name = name;
2317     }
2318 }
2319 public class Mantu {
2320
2321     public static void main(String[]args)
2322     {
2323         // calling no -argument constructor
2324         Human obj = new Human();
2325         Human obj1 = new Human(18, "Mantu");
2326         System.out.println(obj.getName() + " : " + obj.getAge());
2327         System.out.println(obj1.getName() + " : " + obj1.getAge());
2328
2329
2330         //obj.setAge(30);
2331         //obj.setName("Mantu");
2332
2333         // System.out.println(obj.getName() + " : " + obj.getAge());
2334     }
2335
2336 }
2337
2338 45. Naming Convention in java
2339 // Camel Casing

```

```

2340 // class and interface is capital letter
2341 Example:-Calculator:-Calculator
2342 // variable and method smoll letter
2343 Example:- marks,show()
2344 //Constants:- PIE, BRAND
2345 // ShowMyMarks()
2346 //MyDate
2347 // age, DATA, Human()
2348
2349
2350 46.Anonymous Object in java
2351 package Mantu;
2352
2353 class A
2354 {
2355     public A()
2356     {
2357         System.out.println("object.created");
2358     }
2359     public void show()
2360     {
2361         System.out.println(" in A show");
2362     }
2363 }
2364
2365
2366 public class Mantu
2367 {
2368     public static void main(String[]args)
2369     {
2370         A obj = new A();
2371         obj.show();
2372     }
2373 }
2374
2375 ## package Mantu;
2376
2377 class A
2378 {
2379     public A()
2380     {
2381         System.out.println("object.created");
2382     }
2383     public void show()
2384     {
2385         System.out.println(" in A show");
2386     }
2387 }
2388
2389
2390 public class Mantu
2391 {
2392     public static void main(String[]args)
2393     {
2394
2395         new A(); // anonymous object
2396
2397     }
2398 }
2399
2400 ## package Mantu;
2401
2402 class A
2403 {
2404     public A()
2405     {

```

```

2407         System.out.println("object.created");
2408     }
2409     public void show()
2410     {
2411         System.out.println(" in A show");
2412     }
2413 }
2414 }
2415
2416 public class Mantu
2417 {
2418     public static void main(String[]args)
2419     {
2420
2421         new A().show(); // anonymous object
2422
2423
2424     }
2425 }
2426
2427 ## package Mantu;
2428
2429 class A
2430 {
2431     public A()
2432     {
2433         System.out.println("object.created");
2434     }
2435     public void show()
2436     {
2437         System.out.println(" in A show");
2438     }
2439 }
2440 }
2441
2442 public class Mantu
2443 {
2444     public static void main(String[]args)
2445     {
2446
2447         new A().show(); // anonymous object
2448         new A().show();
2449         // created two objects
2450     }
2451 }
2452
2453 47.Need of Inheritance in java
2454
2455 // Parent super class baise
2456 { class Calc
2457     {
2458
2459         {{
2460
2461             add()                // Child sub draive class
2462             sub()                  class Adv Calc
2463             multi()                {
2464             div()                  {{
2465             {
2466             }                      }
2467
2468
2469 48.What is Inheritance in java
2470
2471 package Mantu;
2472
2473 class Calc

```

```

2474 {
2475     public int add(int n1, int n2)
2476     {
2477         return n1 + n2;
2478     }
2479     public int sub(int n1, int n2)
2480     {
2481         return n1 - n2;
2482     }
2483 }
2484
2485
2486
2487 class Mantu{
2488
2489     public static void main(String[]args)
2490     {
2491         Calc obj = new Calc();
2492         int r1 = obj.add(4, 5);
2493         int r2 = obj.sub(7, 3);
2494
2495         System.out.println(r1 + " " + r2);
2496     }
2497 }
2498
2499 49.Single and Multilevel Inheritance in java
2500 package VeryAdvancCalc;
2501
2502     class    Calc
2503     {
2504     public double  power(int n1, int n2 )
2505     {
2506         return Math.pow(n1, n2);
2507     }
2508     public int div(int n1, int n2)
2509     {
2510         return n1/n2;
2511     }
2512 }
2513 public class VeryAdvancCalc extends Calc{
2514
2515     public static void main(String[]args)
2516     {
2517         VeryAdvancCalc obj = new VeryAdvancCalc();
2518         //int r1 = obj.Add(4,5);
2519         //int r2 = obj.sub(7, 3);
2520         //int r3 = obj.Multi(8,3);
2521         int r4 = obj.div(8,4);
2522         double r5 = obj.power(2,3);
2523
2524         System.out.println( r5 +" ");
2525     }
2526 }
2527
2528 50.Multiple Inheritance in java:-
2529 java multiple inheritance does not work
2530
2531 package VeryAdvancCalc;
2532
2533     class A{
2534
2535     }
2536     class B extends A {
2537
2538     }
2539     class C{
2540

```

```

2541     }
2542     public class VeryAdvancCalc extends Calc{
2543
2544         public static void main(String[]args)
2545         {
2546             VeryAdvancCalc obj = new VeryAdvancCalc();
2547             //int r1 = obj.Add(4,5);
2548             //int r2 = obj.sub(7, 3);
2549             //int r3 = obj.Multi(8,3);
2550             int r4 = obj.div(8,4);
2551             double r5 = obj.power(4,3);
2552
2553             System.out.println( r5 +" ");
2554         }
2555     }
2556

```

2557 51. This and Super Method in java

```

2558
2559 package VeryAdvancCalc;
2560
2561 class A
2562 {
2563
2564 }
2565 class B extends A
2566 {
2567     public B()
2568     {
2569         System.out.println("in B");
2570     }
2571 }
2572 public class VeryAdvancCalc{
2573
2574     public static void main(String[]args)
2575     {
2576         B obj = new B();
2577     }
2578 }
2579

```

```

2580 ##. package VeryAdvancCalc;
2581
2582 class A
2583 {
2584     public A() // Constructor
2585     {
2586         System.out.println("in A");
2587     }
2588 }
2589 class B extends A
2590 {
2591     public B() // Constructor
2592     {
2593         System.out.println("in B");
2594     }
2595 }
2596 public class VeryAdvancCalc{
2597
2598     public static void main(String[]args)
2599     {
2600         B obj = new B();
2601     }
2602 }
2603

```

```

2604
2605 ## package VeryAdvancCalc;
2606

```

```

2607 class A

```

```

2608 {
2609     public A() // Constructor
2610     {
2611         System.out.println("in A");
2612     }
2613 }
2614 class B extends A
2615 {
2616     public B() // Constructor
2617     {
2618         System.out.println("in B");
2619     }
2620     public B(int n)
2621     {
2622         System.out.println("int B int ");
2623     }
2624 }
2625 public class VeryAdvancCalc{
2626
2627     public static void main(String[]args)
2628     {
2629         B obj = new B();
2630     }
2631 }
2632
2633 ## package VeryAdvancCalc;
2634
2635 class A
2636 {
2637     public A() // Constructor
2638     {
2639         System.out.println("in A");
2640     }
2641 }
2642 class B extends A
2643 {
2644     public B() // Constructor
2645     {
2646         System.out.println("in B");
2647     }
2648     public B(int n)
2649     {
2650         System.out.println("int B int ");
2651     }
2652 }
2653 public class VeryAdvancCalc{
2654
2655     public static void main(String[]args)
2656     {
2657         B obj = new B(5);
2658     }
2659 }
2660
2661 NOTE:- every constructor in java has a method which a
2662 there menstion that method super();
2663
2664 package VeryAdvancCalc;
2665
2666 class A
2667 {
2668     public A() // Constructor
2669     {
2670         super();
2671         System.out.println("in A");
2672     }
2673     public A(int n)
2674     {

```



```

2675         super();
2676         System.out.println("in A int" );
2677     }
2678 }
2679 class B extends A
2680 {
2681     public B() // Constructor
2682     {
2683         super();
2684         System.out.println("in B");
2685     }
2686     public B(int n)
2687     {
2688         super(n);
2689         System.out.println("int B int ");
2690     }
2691 }
2692 public class VeryAdvancCalc{
2693
2694     public static void main(String[]args)
2695     {
2696         B obj = new B(5);
2697     }
2698 }
2699
2700 ## package VeryAdvancCalc;
2701
2702 class A
2703 {
2704     public A() // Constructor
2705     {
2706         super();
2707         System.out.println("in A");
2708     }
2709     public A(int n)
2710     {
2711         super();
2712         System.out.println("in A int" );
2713     }
2714 }
2715 class B extends A
2716 {
2717     public B() // Constructor
2718     {
2719         super(5);
2720         System.out.println("in B");
2721     }
2722     public B(int n)
2723     {
2724         super(n);
2725         System.out.println("int B int ");
2726     }
2727 }
2728 public class VeryAdvancCalc{
2729
2730     public static void main(String[]args)
2731     {
2732         B obj = new B();
2733     }
2734 }
2735
2736 Note:-every class in java extends the object class
2737 in java
2738
2739
2740 52. Method Overloading in java.
2741

```

```
2742     package VeryAdvancCalc;
2743
2744     class A
2745     {
2746         public void show()
2747         {
2748             System.out.println("in show");
2749         }
2750     }
2751     public class VeryAdvancCalc{
2752
2753         public static void main(String[]args)
2754         {
2755             A obj = new A();
2756             obj.show();
2757         }
2758
2759     ## package Sky;
2760     class A
2761     {
2762         public void show()
2763         {
2764             System.out.println("in show");
2765         }
2766     }
2767
2768     }
2769     class B extends A
2770     {
2771
2772     }
2773
2774     public class Sky {
2775         public static void main(String[]args)
2776         {
2777             B obj = new B();
2778             obj.show();
2779         }
2780     }
2781
2782
2783     ## package Sky;
2784     class A
2785     {
2786         public void show()
2787         {
2788             System.out.println("in A show");
2789         }
2790     }
2791     public void config()
2792     {
2793         System.out.println("in A config");
2794     }
2795     }
2796     class B extends A
2797     {
2798
2799     }
2800
2801     public class Sky {
2802         public static void main(String[]args)
2803         {
2804             B obj = new B();
2805             obj.show();
2806             obj.config();
2807         }
2808     }
```

```
2809
2810 ## package Sky;
2811 class A
2812 {
2813     public void show()
2814     {
2815         System.out.println("in A show");
2816     }
2817     public void config()
2818     {
2819         System.out.println("in A config");
2820     }
2821 }
2822
2823 class B extends A
2824 {
2825     public void show() // overriding in a class
2826     {
2827         System.out.println("in B show");
2828     }
2829 }
2830
2831 public class Sky {
2832     public static void main(String[]args)
2833     {
2834         B obj = new B();
2835         obj.show();
2836         obj.config();
2837     }
2838 }
2839
2840
2841 ## package Sky;
2842 class A
2843 {
2844     public void show()
2845     {
2846         System.out.println("in A show");
2847     }
2848     public void config()
2849     {
2850         System.out.println("in A config");
2851     }
2852 }
2853
2854 class B extends A
2855 {
2856     public void show1() // overriding in a class
2857     {
2858         System.out.println("in B show");
2859     }
2860 }
2861
2862 public class Sky {
2863     public static void main(String[]args)
2864     {
2865         B obj = new B();
2866         obj.show1();
2867         obj.config();
2868     }
2869 }
2870
2871
2872 ## package Sky;
2873 class Calc
2874 {
2875     public int add(int n1, int n2)
```

```

2876     {
2877         return n1+n2;
2878     }
2879 }
2880
2881 }
2882     class AdvCalc extends Calc
2883     {
2884         // public int add(int n1, int n2) // overriding in a class
2885         // {
2886             // return n1+n2+1;
2887         // }
2888     }
2889
2890 public class Sky {
2891     public static void main(String[]args)
2892     {
2893         AdvCalc obj = new AdvCalc();
2894         int r1=obj.add(3,4);
2895         System.out.println(r1);
2896     }
2897
2898 ## package Sky;
2899 class Calc
2900 {
2901     public int add(int n1, int n2)
2902     {
2903         return n1+n2;
2904     }
2905 }
2906
2907 }
2908     class AdvCalc extends Calc
2909     {
2910         public int add(int n1, int n2) // This method is overriding in a class
2911         {
2912             return n1+n2+1;
2913         }
2914     }
2915
2916 public class Sky {
2917     public static void main(String[]args)
2918     {
2919         AdvCalc obj = new AdvCalc();
2920         int r1=obj.add(3,4);
2921         System.out.println(r1);
2922     }
2923
2924 }
2925
2926
2927

```

2928 55. Polymorphism in java
2929 many behaviour

2930
2931 There are two types of polymorphism
2932 (1) Compile time :-behaviour is defined at compile time.
2933 (overloading):-part of compile time because at the time itself.
2934 (2) Run time :- it is behaviour will be run time.(overriding):-

2935
2936 compile time is two methods.

2937
2938 add(int,int)
2939 add(int,int,int) overloading

2940
2941 A
2942 add(int,int)

```
2943
2944
2945
2946
2947             B
2948             add(int,int)
2949
2950
```

2951 56.Dynamic Method Dispatch in java.

```
2952 package Sky;
```

```
2953
```

```
2954 class A    // parents
```

```
2955 {
```

```
2956 public void show()
```

```
2957 {
```

```
2958     System.out.println("in A show");
```

```
2959     }
```

```
2960 }
```

```
2961
```

```
2962 class B extends A    // child
```

```
2963 {
```

```
2964
```

```
2965 }
```

```
2966
```

```
2967 public class Sky {
```

```
2968     public static void main(String[]args)
```

```
2969     {
```

```
2970         B obj = new B();
```

```
2971         obj.show();
```

```
2972     }
```

```
2973
```

```
2974 }
```

```
2975
```

```
2976 ## package Sky;
```

```
2977
```

```
2978
```

```
2979 class A    // parents
```

```
2980 {
```

```
2981 public void show()
```

```
2982 {
```

```
2983     System.out.println("in A show");
```

```
2984     }
```

```
2985 }
```

```
2986
```

```
2987 class B extends A    // child
```

```
2988 {
```

```
2989     public void show()
```

```
2990     {
```

```
2991         System.out.println("in B show");
```

```
2992     }
```

```
2993 }
```

```
2994
```

```
2995 public class Sky {
```

```
2996     public static void main(String[]args)
```

```
2997     {
```

```
2998         A obj = new B();
```

```
2999         obj.show();
```

```
3000
```

```
3001
```

```
3002     }
```

```
3003
```

```
3004 }
```

```
3005
```

```
3006 ##package Sky;
```

```
3007
```

```
3008
```

```
3009 class A    // parents
```

```
3010 {
3011 public void show()
3012 {
3013     System.out.println("in A show");
3014 }
3015 }
3016
3017 class B extends A // child
3018 {
3019     public void show()
3020     {
3021         System.out.println("in B show");
3022     }
3023 }
3024
3025 public class Sky {
3026     public static void main(String[]args)
3027     {
3028         A obj = new A();
3029         obj.show();
3030
3031         obj = new B();
3032         obj.show();
3033     }
3034 }
3035
3036 ## package Sky;
3037
3038
3039 class A // parents
3040 {
3041     public void show()
3042     {
3043         System.out.println("in A show");
3044     }
3045 }
3046
3047 class B extends A // child
3048 {
3049     public void show()
3050     {
3051         System.out.println("in B show");
3052     }
3053 }
3054 class C extends A
3055 {
3056     public void show()
3057     {
3058         System.out.println("in C show");
3059     }
3060 }
3061
3062
3063 public class Sky {
3064     public static void main(String[]args)
3065     {
3066         A obj = new A();
3067         obj.show();
3068
3069         obj = new B();
3070         obj.show();
3071
3072         obj = new C();
3073         obj.show();
3074     }
3075 }
3076
```

```
3077 }
3078
3079 57. Final Keyword in java
3080 Final Keyword:-can be used to with variable, method, class
3081
3082 package Sky;
3083
3084
3085
3086 public class Sky {
3087     public static void main(String[]args)
3088     {
3089
3090         int num = 8;
3091         num = 9;
3092         System.out.println(num);
3093
3094     }
3095
3096 }
3097
3098 ## package Sky;
3099
3100 class Calc
3101 {
3102     public void show()
3103     {
3104         System.out.println("in Calc show");
3105     }
3106     public void add(int a,int b)
3107     {
3108         System.out.println(a+b);
3109     }
3110 }
3111
3112
3113 public class Sky {
3114     public static void main(String[]args)
3115     {
3116         Calc obj = new Calc();
3117         obj.show();
3118         obj.add(8, 12);
3119     }
3120
3121 }
3122
3123
3124 ## package Sky;
3125
3126 class Calc
3127 {
3128     public void show()
3129     {
3130         System.out.println("in Calc show");
3131     }
3132     public void add(int a,int b)
3133     {
3134         System.out.println(a+b);
3135     }
3136 }
3137
3138 class AdvCalc extends Calc
3139 {
3140
3141     }
3142
3143 public class Sky {
```

```

3144     public static void main(String[]args)
3145     {
3146         AdvCalc obj = new AdvCalc();
3147         obj.show();
3148         obj.add(8, 12);
3149     }
3150 }
3151
3152 ## package Sky;
3153
3154     class Calc
3155     {
3156         public void show()
3157         {
3158             System.out.println("By Mantu");
3159         }
3160         public void add(int a,int b)
3161         {
3162             System.out.println(a+b);
3163         }
3164     }
3165
3166     class AdvCalc extends Calc
3167     {
3168         public void show()
3169         {
3170             System.out.println("By puja"); // Overriding
3171         }
3172     }
3173
3174     public class Sky {
3175         public static void main(String[]args)
3176         {
3177             AdvCalc obj = new AdvCalc();
3178             obj.show();
3179             obj.add(8, 12);
3180         }
3181     }
3182
3183 }
3184
3185

```

58.Object class equals to String hashCode in java

```

3186 package Sky;
3187
3188     class Laptop
3189     {
3190         String model;
3191         int price;
3192
3193         public String toString()
3194         {
3195             return "Hey";
3196         }
3197     }
3198
3199
3200     public class Sky {
3201         public static void main(String[]args)
3202         {
3203             Laptop obj = new Laptop();
3204             obj.model = "Hp word ";
3205             obj.price = 50000;
3206
3207             System.out.println(obj.toString());
3208         }
3209     }
3210

```



```
3211     }
3212
3213     ## package Sky;
3214
3215     class Laptop
3216     {
3217         String model;
3218         int price;
3219
3220         public String toString()
3221         {
3222             return "Hey";
3223         }
3224     }
3225
3226     public class Sky {
3227         public static void main(String[] args)
3228         {
3229             Laptop obj = new Laptop();
3230             obj.model = "Hp word ";
3231             obj.price = 50000;
3232
3233             System.out.println(obj);
3234         }
3235     }
3236 }
3237
3238 ## package Sky;
3239
3240 class Laptop
3241 {
3242     String model;
3243     int price;
3244
3245     public String toString()
3246     {
3247         return model + " :" + price;
3248     }
3249 }
3250
3251 public class Sky {
3252     public static void main(String[] args)
3253     {
3254         Laptop obj = new Laptop();
3255         obj.model = "Hp word ";
3256         obj.price = 50000;
3257
3258         System.out.println(obj);
3259     }
3260 }
3261 }
3262
3263
3264 ## package Sky;
3265
3266 class Laptop
3267 {
3268     String model;
3269     int price;
3270
3271     public String toString()
3272     {
3273         return model + " :" + price;
3274     }
3275 }
3276
3277 public class Sky {
```

```

3278     public static void main(String[]args)
3279     {
3280         Laptop obj1 = new Laptop();
3281         obj1.model = "Hp word ";
3282         obj1.price = 50000;
3283
3284         Laptop obj2 = new Laptop();
3285         obj2.model = "Hp word ";
3286         obj2.price = 50000;
3287
3288         boolean result = obj1 == obj2;
3289
3290         System.out.println(result);
3291     }
3292
3293 }
3294
3295 ## package Sky;
3296
3297 class Laptop
3298 {
3299     String model;
3300     int price;
3301
3302     public String toString()
3303     {
3304         return model + " :" + price;
3305     }
3306     public boolean equals(Laptop that)
3307     {
3308         if(this.model.equals(that.model) && this.price == that.price)
3309         {
3310             return true;
3311         }
3312
3313         else
3314
3315             return false;
3316     }
3317 }
3318
3319 public class Sky {
3320     public static void main(String[]args)
3321     {
3322         Laptop obj1 = new Laptop();
3323         obj1.model = "Hp word ";
3324         obj1.price = 50000;
3325
3326         Laptop obj2 = new Laptop();
3327         obj2.model = "Hp word ";
3328         obj2.price = 50000;
3329
3330         boolean result = obj1.equals(obj2);
3331
3332         System.out.println(result);
3333     }
3334 }
3335
3336
3337 59.Upcasting and Downcasting in java.
3338
3339 package Sky;
3340
3341
3342 public class Sky {
3343     public static void main(String[]args)
3344     {

```

```
3345     double d = 4.5;
3346
3347     System.out.println(d);
3348 }
3349
3350 }
3351
3352 ## package Sky;
3353
3354
3355 public class Sky {
3356     public static void main(String[]args)
3357     {
3358         double d = 4.5;
3359         int i = (int) d; // type casting
3360
3361         System.out.println(i);
3362     }
3363
3364 }
3365
3366 ## package Sky;
3367
3368 class A
3369 {
3370     public void show1()
3371     {
3372         System.out.println("in A show");
3373     }
3374
3375     class B extends A
3376     {
3377         public void show2()
3378         {
3379             System.out.println("in B show");
3380         }
3381     }
3382
3383
3384 public class Sky {
3385     public static void main(String[]args)
3386     {
3387         A obj = new A();
3388         obj.show1();
3389     }
3390
3391
3392 ## package Sky;
3393
3394 class A
3395 {
3396     public void show1()
3397     {
3398         System.out.println("in A show");
3399     }
3400
3401     class B extends A
3402     {
3403         public void show2()
3404         {
3405             System.out.println("in B show");
3406         }
3407     }
3408
3409
3410 public class Sky {
3411     public static void main(String[]args)
```

```

3412     {
3413         A obj = (A)new B(); // this method is Upcasting
3414         obj.show1();
3415     }
3416 }
3417
3418 }
3419
3420 ## package Sky;
3421
3422 class A
3423 {
3424     public void show1()
3425     {
3426         System.out.println("in A show");
3427     }
3428 }
3429 class B extends A
3430 {
3431     public void show2()
3432     {
3433         System.out.println("in B show");
3434     }
3435 }
3436
3437
3438 public class Sky {
3439     public static void main(String[]args)
3440     {
3441         A obj = new B();
3442         obj.show1();
3443
3444
3445         B obj1 = (B) obj; // This method is Downcasting
3446         obj1.show2();
3447
3448     }
3449 }
3450 }
3451
3452 60.Wrapper Class in java.
3453
3454 For every primitive type we are going to have class file
3455 int- Integer
3456 char- Charatctor
3457 double- Double
3458
3459 package Sky;
3460
3461 public class Sky {
3462     public static void main(String[]args)
3463     {
3464         int num = 7;
3465         Integer num1 = ( 8); // this called Autoboxing
3466
3467         System.out.println(num1);
3468     }
3469 }
3470 }
3471
3472 ## package Sky;
3473
3474
3475 public class Sky {
3476     public static void main(String[]args)
3477     {
3478         int num = 7;

```

```

3479         Integer num1 = num; // Autoboxing
3480
3481         int num2 = num1.intValue(); // auto- Unboxing
3482
3483         System.out.println(num2);
3484     }
3485
3486 }
3487
3488 ## package Sky;
3489
3490 public class Sky {
3491     public static void main(String[]args)
3492     {
3493         int num = 7;
3494         Integer num1 = num; // Autoboxing
3495
3496
3497         int num2 = num1.intValue(); // auto- Unboxing
3498
3499         System.out.println(num2);
3500
3501         String str = "12";
3502         int num3 = Integer.parseInt(str);
3503
3504         System.out.println(num3*2);
3505
3506     }
3507
3508 }
3509
3510 61.Abstract Keyword in java
3511
3512 package Sky;
3513
3514
3515 class car
3516 {
3517     public void drive()
3518     {
3519
3520     }
3521     public void playMusic()
3522     {
3523         System.out.println("play music");
3524     }
3525 }
3526
3527 ##
3528 public class Sky {
3529     public static void main(String[]args)
3530     {
3531         car obj = new car();
3532         obj.drive();
3533         obj.playMusic();
3534     }
3535
3536 }
3537
3538
3539
3540
3541
3542
3543 public class Sky {
3544     public static void main(String[]args)
3545     {

```

```

3546         int num = 7;
3547         Integer num1 = new Integer( 8); //warning
3548
3549
3550         System.out.println(num1);
3551     }
3552
3553 }
3554
3555 ## package Sky;
3556
3557 public class Sky {
3558     public static void main(String[] args)
3559     {
3560         int num = 7;
3561         Integer num1 = ( 8);
3562
3563
3564         System.out.println(num1);
3565     }
3566
3567 }
3568
3569 62. Inner class in java.
3570     class A
3571     {
3572         int age ;
3573
3574         public void show()
3575         {
3576             System.out.println("in show");
3577         }
3578         class B
3579         {
3580             public void config()
3581             {
3582                 System.out.println("in config");
3583             }
3584         }
3585     }
3586
3587
3588 public class Sky {
3589     public static void main(String[] args)
3590     {
3591         A obj = new A();
3592         obj.show();
3593
3594     }
3595 ## package Sky;
3596
3597 class A
3598 {
3599     int age ;
3600
3601     public void show()
3602     {
3603         System.out.println("in show");
3604     }
3605     class B
3606     {
3607         public void config()
3608         {
3609             System.out.println("in config");
3610         }
3611     }
3612 }

```

```

3613
3614
3615 public class Sky {
3616     public static void main(String[]args)
3617     {
3618         A obj = new A();
3619         obj.show();
3620
3621         A.B obj1 = obj.new B();
3622         obj1.config();
3623     }
3624
3625 }
3626
3627 ## package Sky;
3628
3629 class A
3630 {
3631     int age ;
3632
3633     public void show()
3634     {
3635         System.out.println("in show");
3636     }
3637     static class B
3638     {
3639         public void config()
3640         {
3641             System.out.println("in config");
3642         }
3643     }
3644 }
3645
3646
3647 public class Sky {
3648     public static void main(String[]args)
3649     {
3650         A obj = new A();
3651         obj.show();
3652
3653         A.B obj1 = new A.B();
3654         obj1.config();
3655     }
3656
3657 }
3658
3659 63. Anonymous inner class in java.
3660
3661 package Sky;
3662
3663 class A
3664 {
3665     public void show()
3666     {
3667         System.out.println("in show");
3668     }
3669 }
3670
3671
3672 public class Sky {
3673     public static void main(String[]args)
3674     {
3675         A obj = new A();
3676         obj.show();
3677     }
3678
3679 }

```

```

3680
3681
3682 ## package Sky;
3683
3684 class A
3685 {
3686     public void show()
3687     {
3688         System.out.println("in A show");
3689     }
3690 }
3691 class B extends A
3692 {
3693     public void show()
3694     {
3695         System.out.println("in B show");
3696     }
3697 }
3698
3699 public class Sky {
3700     public static void main(String[]args)
3701     {
3702         A obj = new B();
3703         obj.show();
3704     }
3705 }
3706
3707
3708
3709 64.Abstract and Anonymous inner class.
3710 package Sky;
3711
3712 abstract class A
3713 {
3714     public void show()
3715     {
3716     }
3717 }
3718
3719 class B extends A
3720 {
3721     public void show()
3722     {
3723         System.out.println("in B show");
3724     }
3725 }
3726 public class Sky {
3727     public static void main(String[]args)
3728     {
3729         A obj = new B();
3730         obj.show();
3731     }
3732 }
3733
3734
3735
3736 ## package Sky;
3737
3738 abstract class A
3739 {
3740     public abstract void show()
3741     {
3742     }
3743 }
3744
3745 public class Sky {
3746     public static void main(String[]args)

```



```

3747     {
3748         A obj = new A()
3749         {
3750             public void show()
3751             {
3752                 System.out.println("in new show");
3753             }
3754         };
3755         obj.show();
3756     }
3757 }
3758
3759 }
3760
3761 65.What is Interface in java.
3762 package Sky;
3763
3764 interface A
3765 {
3766     void show();
3767     void config();
3768 }
3769 class B implements A
3770 {
3771     public void show()
3772     {
3773         System.out.println("in show");
3774     }
3775     public void config()
3776     {
3777         System.out.println("in config");
3778     }
3779 }
3780 public class Sky {
3781     public static void main(String[]args)
3782     {
3783         A obj;
3784         obj =new B();
3785         obj.show();
3786         obj.config();
3787     }
3788 }
3789
3790
3791 ## package Sky;
3792
3793 interface A
3794 {
3795     int age=44; // every variable in the interface final and static
3796     String area="Patna";
3797     void show();
3798     void config();
3799 }
3800 class B implements A
3801 {
3802     public void show()
3803     {
3804         System.out.println("in show");
3805     }
3806     public void config()
3807     {
3808         System.out.println("in config");
3809     }
3810 }
3811 public class Sky {
3812     public static void main(String[]args)
3813     {

```

```
3814         A obj;
3815         obj =new B();
3816         obj.show();
3817         obj.config();
3818
3819         System.out.println(A.area);
3820     }
3821
3822 }
3823
3824 66.Need of Interface in java.
3825 package Sky;
3826
3827 class Developer
3828 {
3829     public void devApp()
3830     {
3831         System.out.println("coding");
3832     }
3833 }
3834
3835 public class Sky {
3836     public static void main(String[]args)
3837     {
3838         Developer mantu = new Developer();
3839         mantu.devApp();
3840     }
3841 }
3842
3843
3844 ## package Sky;
3845
3846 class Laptop
3847 {
3848     public void code()
3849     {
3850         System.out.println("code, compile, run");
3851     }
3852 }
3853
3854
3855
3856 class Developer
3857 {
3858     public void devApp(Laptop lap)
3859     {
3860         lap.code();
3861     }
3862 }
3863
3864 public class Sky {
3865     public static void main(String[]args)
3866     {
3867         Laptop lap = new Laptop();
3868
3869
3870
3871         Developer mantu = new Developer();
3872         mantu.devApp(lap);
3873     }
3874 }
3875
3876
3877 67.
3878
3879
3880 68.What is Enum in java.
```

```
3881     package Sky;
3882
3883     enum Status
3884     {
3885         Running,Failed,Pending,Success;
3886     }
3887
3888     public class Sky {
3889         public static void main(String[]args)
3890         {
3891             int i = 5;
3892             Status s = Status.Running;
3893             System.out.println(s);
3894         }
3895     }
3896
3897     ## package Sky;
3898
3899     enum Status
3900     {
3901         Running,Failed,Pending,Success;
3902     }
3903
3904     public class Sky {
3905         public static void main(String[]args)
3906         {
3907             int i = 5;
3908             Status s = Status.Failed;
3909             System.out.println(s);
3910         }
3911     }
3912
3913
3914     ##package Sky;
3915
3916     enum Status
3917     {
3918         Running,Failed,Pending,Success;
3919     }
3920
3921     public class Sky {
3922         public static void main(String[]args)
3923         {
3924             int i = 5;
3925             Status s = Status.Running;
3926             System.out.println(s.ordinal());
3927         }
3928     }
3929
3930
3931     ## package Sky;
3932
3933     enum Status
3934     {
3935         Running,Failed,Pending,Success;
3936     }
3937
3938     public class Sky {
3939         public static void main(String[]args)
3940         {
3941             int i = 5;
3942             Status[] ss = Status.values();
3943             System.out.println(ss[0]);
3944         }
3945     }
3946
3947     ## package Sky;
```

```

3948
3949 enum Status
3950 {
3951     Running,Failed,Pending,Success;
3952 }
3953
3954 public class Sky {
3955     public static void main(String[]args)
3956     {
3957         int i = 5;
3958         Status[] ss = Status.values();
3959
3960         for(Status s: ss)
3961         {
3962             System.out.println(s);
3963         }
3964     }
3965
3966 }
3967
3968 69.Enum if and Switch in java.
3969 package Sky;
3970
3971 enum Status
3972 {
3973     Running,Failed,Pending,Success;
3974 }
3975
3976 public class Sky {
3977     public static void main(String[]args)
3978     {
3979         Status s = Status.Pending;
3980
3981         if(s == Status.Running)
3982             System.out.println("All Good");
3983         else if(s == Status.Failed)
3984             System.out.println("Try Again");
3985         else if(s ==Status.Pending)
3986             System.out.println("Please Wait");
3987         else
3988             System.out.println("Done");
3989     }
3990
3991 }
3992
3993 ## package Sky;
3994
3995 enum Status
3996 {
3997     Running,Failed,Pending,Success;
3998 }
3999
4000 public class Sky {
4001     public static void main(String[]args)
4002     {
4003         Status s = Status.Running;
4004
4005         if(s == Status.Running)
4006             System.out.println("All Good");
4007         else if(s == Status.Failed)
4008             System.out.println("Try Again");
4009         else if(s ==Status.Pending)
4010             System.out.println("Please Wait");
4011         else
4012             System.out.println("Done");
4013     }
4014

```

```

4015 }
4016
4017
4018
4019
4020 HTML.
4021 Level 0
4022 1. IDE or Code Editor
4023 1.What is IDE
4024 2. Need of IDE
4025 3. IDE Selection
4026 4. Installation Extension
4027 vs code Extensions
4028
4029 2.Wesbsite Components and Fundamentals.
4030
4031 1.Client Side vs Server Side
4032 2. FrontEnd/BackEnd/FulStack
4033 3. Role of Browser
4034 4. HTML
4035 5. CSS
4036 6. JS
4037
4038
4039 1. IDE or Code Editor.
4040
4041 1.1 What is IDE
4042 1. IDE Stands for Integrated Development Environment.
4043 2. Software suite that consolidates basic tools required for Software development.
4044
4045 3. Central hub for coding, finding problems, and testing.
4046 4. Designed to improve developer efficncy.
4047
4048 1.2 Need of IDE
4049 1. Streamlines development.
4050 2. Increases productivity.
4051 3. Simplifies complex tasks.
4052 4. Offers a unified workspace.
4053 5. IDE Features
4054
4055 1. code Autocomplete
4056 2. Syntax Highlighting
4057 3. Vesion Control
4058 4. Error Checking
4059
4060 1.3 IDE Selection
4061 1. Sublime Text
4062 2. Atom
4063 3. VS Code
4064 4. Github CodeSpaces
4065
4066 1.4 Installation & Setup
4067
4068 1.Search VS Code
4069
4070 1.5 VSCode Extensions
4071 1. Live Server
4072 2. Prettier
4073
4074 Lelvel 0.
4075 2. Website components And Fundamentals
4076
4077 2.1 Client Side vs Server Side.
4078
4079
4080
4081

```

```

4082
4083 Languages           primarily java   PHP,Python,java
4084                      script,HTML,CSS.  Node.js,etc
4085
4086 Main Job             Makes clicks   Manages
4087                      and scrolls    saved information
4088                      work
4089
4090 Access Level         can't access   can read/write file
4091                      server data    interact with
4092
4093 Speed               Quicker for UI   Slower due to
4094                      changes         network latency
4095
4096
4097 2.2 FrontEnd/BackEnd/FullStack
4098
4099 2.3 Role of Browser
4100 1. Displays web page;- Turns HTML code into what you see on screen.
4101 2. User Clicks:- Helps you interact with the web page.
4102 3. Updates Content:- Allows changes to the page using javaScript.
4103 4. Loads Files:- Gets HTML, images,etc,from the server.
4104
4105 2.4 HTML (Hypertext Markup Language)
4106
4107 1. Structure:- Sets up the layout.
4108 2. Content:- Adds text, images, links.
4109 3. Tags:- Uses elements like <p>,<a>.
4110 4. Hierachy:- Organizes elements in a tree.
4111
4112 2.5 CSS (cascading style sheets)
4113
4114 1. Style:- Sets the look and feel.
4115 2. colors & Fonts:- Customize text and background.
4116 3. Layout:- Controls position and size.
4117 4. Selectors:- Targets specific HTML elements.
4118
4119 2.6 Js (Java Script)
4120
4121 1. javaScript has nothing to do with java.
4122 2. Actions:- Enables interactivity.
4123 3. Updates:- Alters page without reloading.
4124 4. Events:- Responds to user actions.
4125 5. Data:- Fetches and sends info to server.
4126
4127
4128 LEVEL 1
4129 HTML Basics
4130 1. Starting up
4131 1. First File using Text Editor
4132 2. File Extensions
4133 3. Opening the project in VsCode
4134 4. Index.html
4135
4136 2.Basics of HTML
4137 1. What are Tags
4138 2. Using Emmett ! to generate code
4139 3. Basic HTML page
4140 4. MDN Documentation
4141 5. Comments
4142 6. Case Sensitivity
4143
4144 1.1 Firs file using Text Editor
4145
4146 1. Create a folder with name First Project on your Desktop.
4147 2. Open Notepad.
4148 3. Create a file and sace it as index.html

```

4149 **4. Copy Sample code**
4150 **5. Open Browser and Check.**
4151
4152